

Loja de Música — Relatório Técnico da 2.^a Iteração

Dinis Rodrigues 1250853, Guilherme Martins 1251012, Mateus Rodrigues
1251275, Rodrigo Santos 1251419
Grupo 3, Turma 1DA

Fundamentos de Desenvolvimento de Software, 2025/2026

Resumo. Este relatório descreve o design e a implementação da segunda iteração de uma aplicação CLI em C++ para a gestão de uma loja de música digital. A arquitetura do sistema foi modelada através de diagramas UML de implantação, componentes e sequência, adotando uma estrutura em camadas para assegurar a separação de conceitos. A atribuição de responsabilidades foi fundamentada nos princípios GRASP, otimizando a coesão e o acoplamento entre as entidades lógicas. A robustez do software é garantida por validações sintáticas rigorosas e um mecanismo de tratamento de exceções customizadas capturadas na interface.

Palavras-chave: Engenharia de Software, Arquitetura em Camadas, Princípios GRASP, Robustez de Código, Persistência CSV.

1 Introdução

O presente relatório técnico descreve a fase de desenho e implementação da aplicação desenvolvida em C++ para a gestão e comercialização de um catálogo de música digital. Após uma primeira iteração dedicada ao levantamento de requisitos operacionais e à especificação do sistema, esta segunda fase concretiza as decisões arquiteturais necessárias para garantir o funcionamento estável da aplicação através de uma interface de linha de comandos (CLI) estável. O sistema visa servir dois perfis distintos de utilizadores: o Administrador, com permissões para gerir o acervo de artistas e álbuns, e o Cliente, capacitado para navegar no catálogo, gerir o seu carrinho de compras e atribuir classificações.

O principal desafio técnico desta iteração consistiu em desenhar uma arquitetura robusta e descentralizada sem recorrer a sistemas de bases de dados tradicionais. Para o efeito, a aplicação foi estruturada em quatro camadas lógicas distintas (Apresentação, Controlo, Domínio e Persistência), com a atribuição de responsabilidades a ser guiada pelos padrões GRASP (*General Responsibility Assignment Software Patterns*). Adicionalmente, o desenvolvimento focou-se na

blindagem da interface contra inputs inválidos e na garantia de mecanismos de tratamento de erros, assegurando que o estado do domínio seja preservado de forma segura entre execuções.

O restante documento encontra-se organizado da seguinte forma: o Capítulo 2 detalha a arquitetura física e lógica do sistema, bem como a organização do código-fonte; o Capítulo 3 apresenta a modelação dinâmica através de diagramas de sequência; o Capítulo 4 aborda a aplicação prática dos princípios GRASP; os Capítulos 5 e 6 expõem as boas práticas de programação orientada a objetos e os mecanismos de robustez implementados; por fim, o Capítulo 7 apresenta as conclusões do projeto e as perspectivas de trabalho futuro.

2 Arquitetura e Design do Sistema

Esta secção descreve a estruturação do sistema de gestão da loja de música sob duas perspetivas fundamentais: a física, que detalha a infraestrutura de hardware onde a aplicação é executada, e a lógica, que mapeia a organização interna do código em camadas independentes. Adicionalmente, apresenta-se a topologia da interface de utilizador, evidenciando o fluxo de navegação hierárquica dos menus disponibilizados.

2.1 Arquitetura Física

A arquitetura física do sistema é modelada através de um Diagrama de Implantação (*Deployment Diagram*). A aplicação foi desenvolvida como uma solução *standalone*, o que significa que todos os componentes executáveis, estruturas de dados e ficheiros de persistência residem e correm localmente num único nó de computação (PC). Não existem dependências de servidores de bases de dados externos ou camadas de rede intermédias, simplificando a sua distribuição e execução direta no sistema operativo do utilizador final.

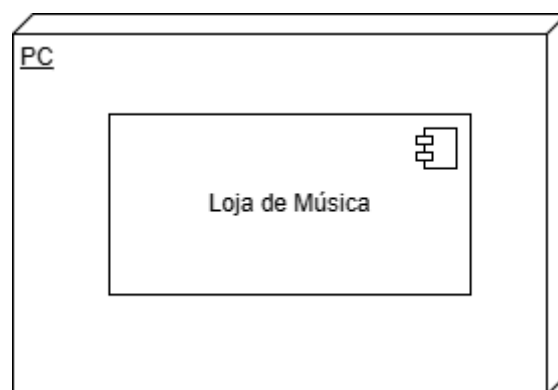


Figure 1 - Arquitetura Física

2.2 Arquitetura Lógica

A arquitetura lógica define a organização estrutural do código-fonte e foi modelada através de um Diagrama de Componentes (*Component Diagram*). Adotou-se um padrão arquitetural estruturado em quatro camadas lógicas para assegurar a separação de conceitos e o baixo acoplamento entre os módulos:

- **Camada de Apresentação (View):** Representada pelos ficheiros Menus e Utils. Controla exclusivamente a renderização do texto no terminal e a captura sintática dos dados introduzidos pelo utilizador.
- **Camada de Controlo (Controller):** Materializada na classe ManagerCatalogo. Atua como intermediária entre a interface pública e as entidades, coordenando a execução das funcionalidades do sistema sem expor diretamente o domínio à UI.
- **Camada de Domínio / Modelo (Model):** Composta pelas entidades de negócio (Artista, Album, Carrinho, Venda, Rating). Estas classes encapsulam os atributos privados e as regras de consistência lógica do acervo musical.
- **Camada de Persistência (Persistence):** Isolada na classe DataManager, sendo a única responsável por gerir a pasta data/ e efetuar o processamento de leitura e escrita nos ficheiros CSV locais.

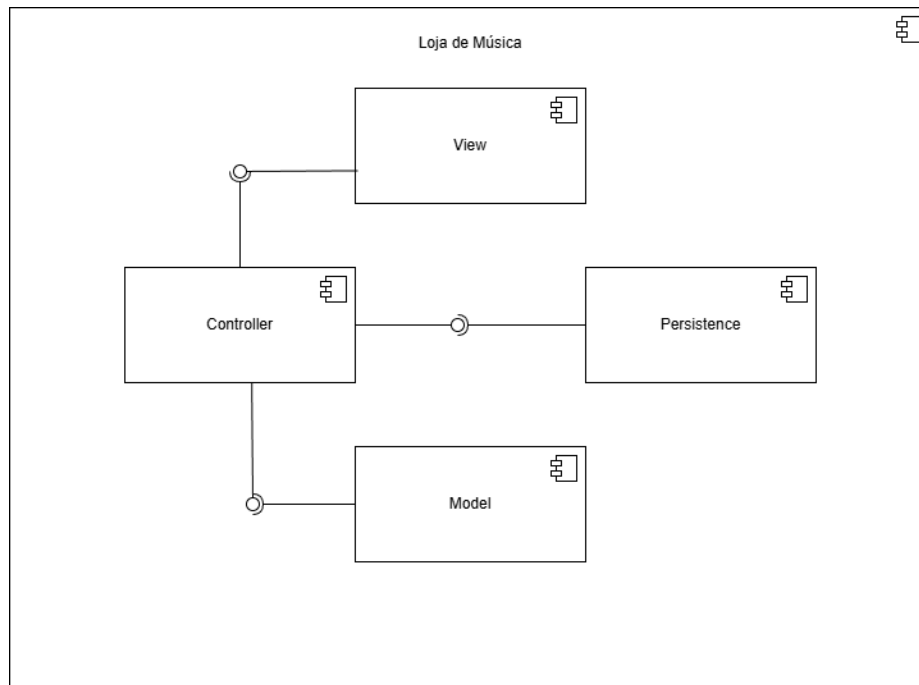


Figure 2 - Arquitetura Lógica

2.3 Organização de Código

A organização do código-fonte reflete a separação de responsabilidades adotada no desenvolvimento do sistema, agrupando os ficheiros de cabeçalho (.h) e de implementação (.cpp) em pacotes lógicos que representam as diferentes camadas da aplicação. Esta abordagem garante o baixo acoplamento e facilita a manutenção do software.

O fluxo estrutural está organizado em torno dos seguintes componentes e módulos principais:

- **Ficheiro Principal (main.cpp):** Atua como o ponto de entrada do programa, sendo responsável por instanciar o controlador global e delegar o arranque da interface ao menu principal.
- **Módulo de Interface (views):** Contém a lógica dos menus textuais (Menus) e as funções utilitárias de validação de dados por teclado (Utils). É a camada que interage diretamente com o utilizador (Cliente ou Administrador).
- **Módulo de Controlo (controllers):** Centralizado na classe ManagerCatalogo, este pacote gere as operações de negócio, coordenando o fluxo de dados entre a interface, a persistência e os modelos.
- **Módulo de Persistência (persistence):** Representado pela classe DataManager, isola as rotinas de leitura e escrita dos ficheiros CSV no disco, carregando e salvando o estado do catálogo.

- **Módulo de Domínio (models):** Agrupa as entidades fundamentais do sistema (Artista, Album, Carrinho, Venda e Rating), que definem as estruturas de dados e os objetos de negócio da aplicação.
- **Módulo de Exceções (exceptions):** Concentra a gestão de erros através de classes dedicadas como InvalidDataException, permitindo a captura e o tratamento robusto de falhas em tempo de execução.

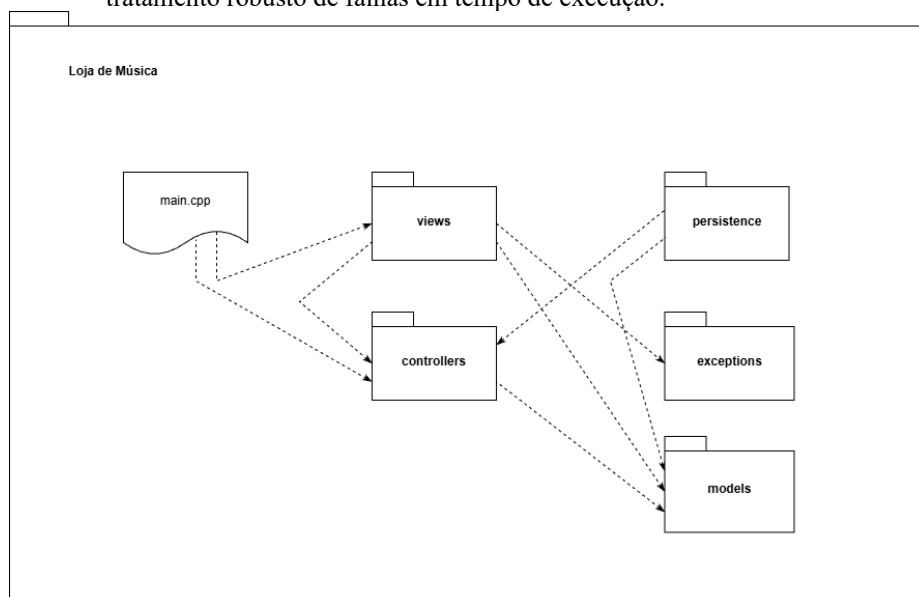


Figure 3 - Organização de Código

3 Modelação de Casos de Uso

A modelação dinâmica do sistema descreve o comportamento do software em tempo de execução, demonstrando como os diferentes componentes das camadas arquiteturais interagem entre si para satisfazer os requisitos funcionais. De forma a ilustrar os fluxos mais críticos e os principais mecanismos arquiteturais da aplicação (Criação, Associação e Robustez), foram selecionados três Casos de Uso (UC) fundamentais, detalhados seguidamente através de Diagramas de Sequência UML.

3.1 UC01 – Autenticação do Administrador (Mecanismo de Robustez)

Este caso de uso ilustra o fluxo de controlo e a validação de segurança no acesso à área de gestão. O diagrama evidencia o tratamento de exceções da aplicação: caso a palavra-passe introduzida pelo utilizador seja incorreta, o sistema instância e lança

uma exceção personalizada (InvalidDataException), que é imediatamente capturada pelo bloco catch da interface (Menus.cpp) para bloquear o acesso e exibir um alerta de segurança no terminal.

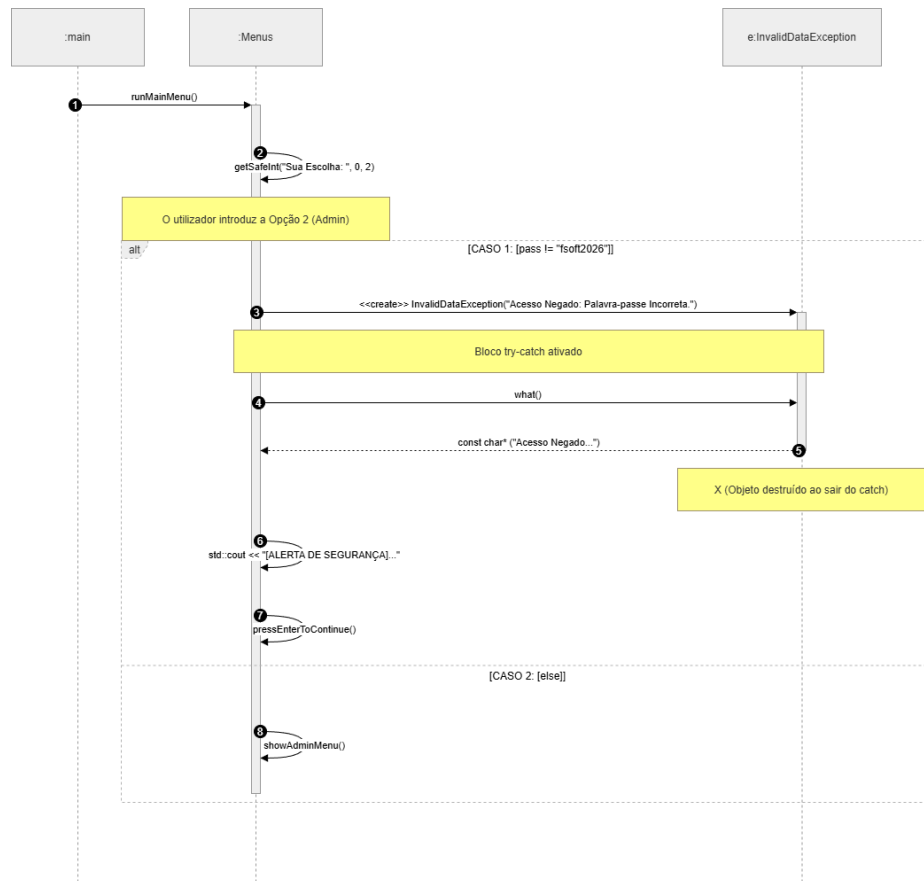


Figure 4 - Autenticação do Administrador (Mecanismo de Robustez)

3.2 UC02 – Adicionar Artista ao Catálogo (Mecanismo de Criação)

Este caso de uso representa a operação essencial de escrita e expansão do acervo da loja por parte do Administrador. O fluxo demonstra a viagem do dado desde a captura de texto na interface (Menus.cpp), passando pela invocação do método no controlador (ManagerCatalogo::adicionarArtista), até à instanciação física do objeto da classe Artista através do seu construtor e respetiva inserção no vetor dinâmico (`std::vector<Artista>`) em memória RAM.

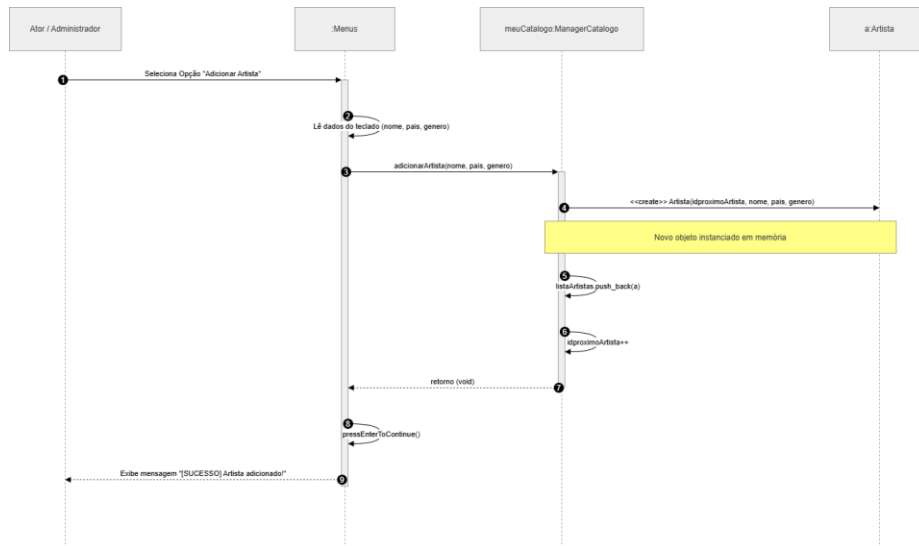


Figure 5 - Adicionar Artista ao Catálogo (Mecanismo de Criação)

3.3 UC03 – Adicionar Álbum ao Carrinho (Mecanismo de Associação por Ponteiros)

Este caso de uso detalha a jornada do Cliente ao selecionar um produto do catálogo para o seu carrinho temporário. Este fluxo é o mais relevante do ponto de vista da gestão de memória, pois demonstra a pesquisa de entidades e a resolução de endereços: a interface obtém o apontador lógico direto do objeto original (Album*) gerido pelo controlador e transmite-o de forma segura para ser armazenado no vetor de itens da classe Carrinho, evitando a duplicação desnecessária de dados.

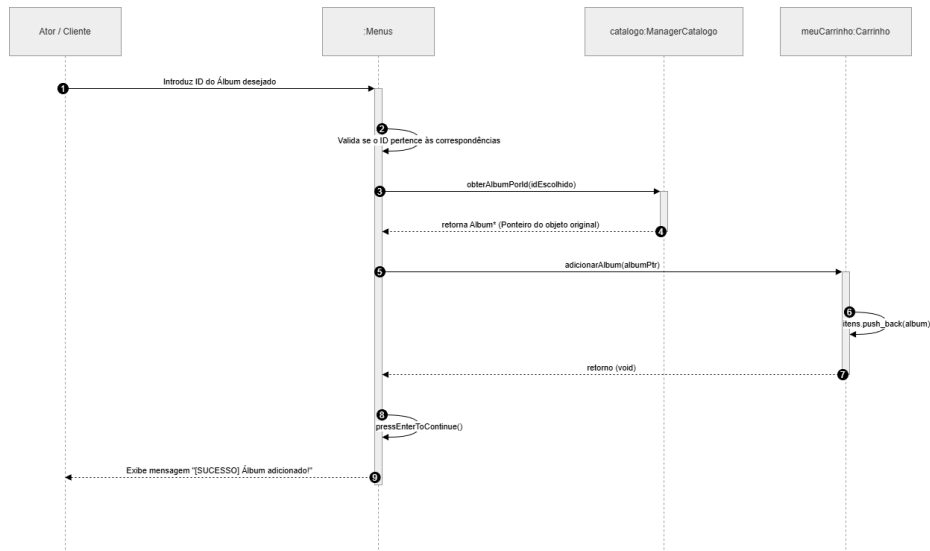


Figure 6 - Adicionar Álbum ao Carrinho (Mecanismo de Associação por Ponteiros)

4 Atribuição de Responsabilidades (Princípios GRASP)

A atribuição de responsabilidades é um dos passos mais críticos no design de software orientado a objetos. Para guiar a distribuição de funções entre as classes da aplicação, foram aplicados os padrões **GRASP** (*General Responsibility Assignment Software Patterns*). Esta abordagem garante que o sistema permaneça modular, fácil de estender e tolerante a alterações. Seguidamente, detalha-se a aplicação prática destes princípios no projeto.

4.1 Conceitos e Exemplos Práticos

4.1.1 Information Expert (Especialista na Informação)

O padrão *Information Expert* determina que a responsabilidade de executar uma determinada operação deve ser atribuída à classe que possui a informação necessária para a realizar.

- **Aplicação no Projeto:** O cálculo do valor total de uma transação foi atribuído exclusivamente à classe Carrinho através do método `calcularTotal()`. Como a classe Carrinho encapsula o vetor privado `itens` (que armazena os apontadores `Album*` adicionados), ela é a autoridade máxima e o "Especialista" que detém os dados dos preços de cada álbum (`album->preco1()`). Isolar esta lógica no Carrinho evita que a camada de interface pública (`Menus.cpp`) tenha de iterar e processar diretamente os dados internos do domínio, respeitando o encapsulamento.

```
float Carrinho::calcularTotal() const {
    float total = 0.0f;
    for (const auto& album : Album * const & : itens) {
        total += album->preco1();
    }
    return total;
}
```

Figure 7 - Information Expert

4.1.2 Creator (Criador)

O padrão *Creator* ajuda a decidir que classe deve ser responsável pela instanciação de novos objetos. Uma classe B deve ter a responsabilidade de criar uma classe A se B agrega, contém, registra ou utiliza de forma muito próxima os objetos de A.

- **Aplicação no Projeto:** A classe ManagerCatalogo atua como o *Creator* oficial das entidades de negócio do sistema (objetos Artista e Album).
 - No método adicionarArtista(), a classe ManagerCatalogo instancia diretamente o construtor de Artista e adiciona-o ao seu vetor listaArtistas.
 - No método adicionarAlbum(), o controlador valida primeiro se o id_artista associado existe e, em caso afirmativo, instancia o objeto Album inserindo-o em listaAlbuns.

Como o ManagerCatalogo é a estrutura que armazena, gere e controla o ciclo de vida completo do acervo em memória RAM, ele preenche todos os requisitos do padrão *Creator*.

```
void ManagerCatalogo::adicionarArtista(const std::string& nome, const std::string& pais, const std::string& genero) {
    Artista a(idproximoArtista, nome, pais, genero);
    listaArtistas.push_back(a);
    idproximoArtista++;
}
```

Figure 8 - Creator

4.1.3 Controller (Controlador)

O padrão *Controller* define o primeiro objeto além da camada de interface que recebe e coordena as mensagens do sistema. Ele funciona como uma fachada de controle que separa as interações visuais da lógica de negócio pura.

- **Aplicação no Projeto:** A classe ManagerCatalogo assume o papel de *Controller* central do software. O ficheiro Menus.cpp (camada de Apresentação/UI) nunca manipula os vetores de dados diretamente, nem toma decisões de consistência de estado. Sempre que o utilizador submete uma ação no terminal — como remover um artigo ou efetuar uma pesquisa — a interface captura o input e delega imediatamente a execução ao

controlador através de métodos como `removerAlbum(id)` ou `pesquisarPorGenero(genero)`. Isto impede o acoplamento direto da UI com o domínio.

```
if (catalogo.removerAlbum(idParaApagar, correspondencias[0].id_album())) {
    std::cout << "\n[SUCESSO] Álbum '" << correspondencias[0].titulo1() << "' removido com sucesso!\n";
}
else {
    std::cout << "\n[INFO] Foram encontrados múltiplos álbuns. Selecione o ID do que deseja apagar:\n";
}
```

Figure 9 - Controller

4.1.4 Alta Coesão (High Cohesion) e Baixo Acoplamento (Low Coupling)

Estes dois princípios funcionam como métricas avaliativas de design. A **Alta Coesão** garante que as responsabilidades de um elemento estejam fortemente focadas e relacionadas, enquanto o **Baixo Acoplamento** assegura que as classes não dependam excessivamente umas das outras, facilitando alterações isoladas.

- **Alta Coesão no Projeto:** Cada classe possui um propósito único e bem delimitado. A classe `DataManager` foca-se estritamente na leitura e escrita de ficheiros CSV no disco (Persistência); as funções em `Utils.cpp` dedicam-se apenas à validação sintática de entradas por teclado (Robustez da UI); e as classes de entidade (`Album`, `Artista`) limitam-se a modelar os atributos e os métodos de acesso públicos (*getters* e *setters*). Não existem classes "divinas" que acumulam funções díspares.
- **Baixo Acoplamento no Projeto:** O acoplamento foi minimizado através de duas estratégias técnicas:
 1. **Associação por Ponteiros:** A classe `Carrinho` guarda referências dinâmicas (`Album*`) e não cópias físicas dos álbuns. Se os dados de um álbum mudarem no catálogo geral, o carrinho reflete isso automaticamente sem necessidade de sincronização complexa.
 2. **Relacionamento por Identificadores (IDs):** A entidade `Album` não contém um objeto `Artista` embutido, mas sim um inteiro primitivo `id_artista`. Isto reduz a dependência estrutural em cadeia na memória e permite que as classes sejam compiladas de forma muito mais independente, facilitando a manutenção e prevenindo fugas de memória.

```
int getSafeInt(std::string prompt, int min, int max) {
    int value;
    while (true) {
        value = getSafeInt(prompt); // Chama a função acima
        if (value >= min && value <= max) {
            return value;
        }
        std::cout << "[ERRO] Opção inválida. Intervalo permitido: " << min << " - " << max << std::endl;
    }
}
```

Figure 10 - High Cohesion

```
void Carrinho::adicionarAlbum(Album* album) {  
    if (album != nullptr) {  
        itens.push_back(album);  
    }  
}
```

Figure 11 - Low Coupling

5 Implementação e Boas Práticas de POO

A transição do paradigma procedimental para a Programação Orientada a Objetos (POO) exige a adoção de um conjunto de regras estruturais e de design que garantam a legibilidade, modularidade e segurança do código-fonte. Esta secção detalha as práticas recomendadas e os padrões de codificação implementados pelo grupo no desenvolvimento do sistema.

5.1 Regras Estruturais (Separação de Ficheiros)

De forma a respeitar os padrões de compilação da linguagem C++ e a organizar o projeto de forma profissional, adotou-se uma separação estrita entre as declarações das classes e as suas respetivas implementações:

- **Diretoria Headers/ (.h):** Contém os ficheiros de cabeçalho. Estes funcionam como o "contrato público" ou interface de cada classe, delimitando os protótipos dos métodos, assinaturas de funções e membros de dados, sem expor a lógica algorítmica.
- **Diretoria Sources/ (.cpp):** Contém os ficheiros de implementação. Constituem as unidades de tradução onde a lógica de negócio é efetivamente desenvolvida e codificada.

Esta dissociação física reduz o tempo de recompilação do projeto e resolve dependências circulares através de inclusões limpas.

```
#include "../../Headers/views/Menus.h"
#include "../../Headers/views/Uteis.h"
#include "../../Headers/exceptions/InvalidDataException.h"
#include "../../Headers/controllers/ManagerCatalogo.h"
#include "../../Headers/models/Album.h"
#include "../../Headers/models/Carrinho.h"
#include "../../Headers/persistence/DataManager.h"
```

Figure 12 - Separação de Ficheiros

5.2 Encapsulamento de Dados

O princípio do encapsulamento foi aplicado rigorosamente em todas as entidades do sistema para proteger o estado interno dos objetos contra manipulações externas indevidas ou estados corrompidos.

- **Atributos Privados (private):** Todos os membros de dados (variáveis de instância) das classes do domínio, como idAlbum, preco, listaArtistas ou itens, foram declarados com visibilidade privada.
- **Métodos Públicos (public):** O acesso e a modificação das propriedades dos objetos são controlados de forma sanitizada através de métodos de acesso (*getters*) e de modificação (*setters*).
- **Garantia de Modificadores const (Const-Correctness):** Todos os métodos que realizam apenas operações de leitura e consulta de dados foram blindados com o modificador const no final da sua assinatura. Isto garante, em tempo de compilação, que o método não alterará os atributos do objeto, promovendo a segurança e permitindo a otimização por parte do compilador.

```
float Venda::getValorTotal() const {
    return valorTotal;
}
```

Figure 14 - Encapsulamento de Dados

```
int Venda::getIdVenda() const {
    return idVenda;
}
```

Figure 13 - Encapsulamento de Dados

5.3 Convenções de Nomenclatura

A consistência estilística é fundamental para o trabalho colaborativo em engenharia de software. O grupo alinhou o desenvolvimento sob as regras universais de escrita de código, corrigindo desvios iniciais de nomenclatura para uniformizar o projeto:

- **PascalCase (UpperCamelCase):** Utilizado exclusivamente para a identificação de Classes, Estruturas e Exceções do sistema. Todas as palavras iniciam com letra maiúscula (ex: ManagerCatalogo, DataManager, InvalidDataException). Durante o refactoring do código, o grupo realizou a correção crítica da classe de gestão do carrinho de compras, alterando a nomenclatura original em minúsculas (carrinho) para a forma correta capitalizada (Carrinho), integrando-a nos standards do projeto.
- **camelCase (LowerCamelCase):** Utilizado para a nomenclatura de métodos, funções e variáveis locais. O primeiro termo inicia em minúsculas e as palavras subsequentes iniciam em maiúsculas (ex: adicionarArtista, calcularTotal, idParaApagar).
- **Sufixos Utilitários:** Adopção de indexações numéricas claras ou sufixos de tipo (como idStr ou albumPtr) para identificar de imediato a natureza das variáveis durante a leitura do fluxo de execução.

6 Robustez, Validação e Gestão de Erros

A estabilidade e a resiliência em tempo de execução (*runtime*) são atributos fundamentais de uma aplicação de software industrial. Um sistema robusto deve ser capaz de detetar anomalias, sanitizar canais de entrada de dados e recuperar de estados abstratos de falha sem que ocorra a interrupção abrupta do programa (*crash*). Na presente arquitetura, a tolerância a falhas foi implementada em duas frentes complementares: a validação sintática preventiva na camada de interface e o desvio estrutural do fluxo por meio de exceções orientadas a objetos.

6.1 Validação Sintática (Proteção do Fluxo de Entrada)

Um dos problemas clássicos de robustez na linguagem C++ reside na manipulação do fluxo padrão de entrada (`std::cin`). Quando a aplicação espera a introdução de um tipo primitivo numérico (como um identificador `int` ou um preço `float`) e o utilizador introduz caracteres alfabéticos ou símbolos sintaticamente inválidos, o canal de leitura entra em estado de falha (*fail state*), bloqueando a execução ou gerando ciclos infinitos.

Para blindar completamente o terminal contra esta vulnerabilidade, foi desenvolvida a biblioteca utilitária `Utils.cpp`. O fluxo é controlado recursivamente através do método sobrecarregado `getSafeInt()`, cuja assinatura e comportamento garantem a integridade dos dados:

```

int getSafeInt(std::string prompt) {
    int value;
    while (true) {
        std::cout << prompt;
        if (std::cin >> value) {
            return value;
        } else {
            std::cout << "[ERRO] Entrada inválida! Por favor, introduza um numero." << std::endl;
            std::cin.clear();
            std::cin.ignore(nstd::numeric_limits<std::streamsize>::max(), delim '\n');
        }
    }
}

```

Figure 15 - Proteção do Fluxo de Entrada

Adicionalmente, o polimorfismo do método permite validar os limites lógicos do negócio, garantindo que o inteiro capturado se situa estritamente dentro de um intervalo válido (como as opções de menu de 0 a 2 ou anos civis válidos):

```

int getSafeInt(std::string prompt, int min, int max) {
    int value;
    while (true) {
        value = getSafeInt(prompt); // Chama a função acima
        if (value >= min && value <= max) {
            return value;
        }
        std::cout << "[ERRO] Opção inválida. Intervalo permitido: " << min << " - " << max << std::endl;
    }
}

```

Figure 16 - Proteção do Fluxo de Entrada

6.2 Mecanismos de Exceções Customizadas

Quando a validação sintática não é suficiente para prever anomalias de negócio (por exemplo, violações de segurança ou violações de regras lógicas do domínio), o sistema recorre ao lançamento de exceções. Em vez de utilizar as classes genéricas da biblioteca padrão (std::exception), o grupo desenvolveu classes de exceções personalizadas para enriquecer a semântica do rastreio de erros:

6.2.1 InvalidDataException (Gestão de Dados Globais e Autenticação)

Esta classe foi concebida para sinalizar incongruências estruturais em operações críticas, tais como falhas de autenticação administrativa. Ela herda o comportamento base de tratamento de erros e substitui o método virtual what() com o qualificador noexcept, assegurando que a extração da mensagem não lançará novas exceções colaterais.

```
#include "../Headers/exceptions/InvalidDataException.h"

InvalidDataException::InvalidDataException(const std::string& msg) : message(msg) {}

const char* InvalidDataException::what() const noexcept {
    return message.c_str();
}
```

Figure 17 - Gestão de Dados Globais e Autenticação

6.2.2 InvalidRatingException (Validação de Domínio)

Implementada para assegurar a consistência métrica das avaliações do catálogo musical, esta exceção é disparada diretamente na classe de domínio Rating.cpp sempre que uma classificação ultrapassa a escala padrão da aplicação (1 a 5 estrelas).

```
void Rating::setPontuacao(int pont) {
    if (pont < 1 || pont > 5) {
        throw InvalidRatingException(message: "Erro de Validação: A pontuação tem de ser entre 1 e 5 estrelas!");
    }
    pontuacao = pont;
}
```

Figure 18 - Validação de Domínio

6.3 Fluxo de Captura (try-catch)

O disparo de uma exceção (throw) desfaz a pilha de chamadas (*stack unwinding*) à procura de um bloco de captura compatível. Se nenhuma sub-rotina a interceptar, a execução aborta de forma destrutiva. Na presente aplicação, a camada de *View* (Menus.cpp) atua como a barreira de contenção final, implementando blocos try-catch nos pontos operacionais mais sensíveis.

O exemplo mais expressivo deste fluxo dinâmico ocorre no login administrativo contido em Menus.cpp. Quando uma palavra-passe incorreta é inserida, a exceção é interceptada imediatamente no escopo local, impedindo o encerramento da aplicação e permitindo que o utilizador tente novamente de forma segura:

```
try {
    if (pass != "fsoft2026") {
        throw InvalidDataException(msg: "Acesso Negado: Palavra-passe Incorreta.");
    }
    showAdminMenu(& meuCatalogo);
} catch (const InvalidDataException& e) {
    std::cout << "\n[ALERTA DE SEGURANÇA] " << e.what() << std::endl;
    pressEnterToContinue();
}
```

Figure 19 - Fluxo de Captura

Este encapsulamento dinâmico garante que as falhas operacionais sejam tratadas como eventos controlados do sistema, elevando o índice de confiabilidade e robustez do software entregue.

7 Conclusões e Trabalho Futuro

O desenvolvimento do presente sistema de gestão de catálogo musical permitiu consolidar com sucesso os conceitos fundamentais de Programação Orientada a Objetos (POO) e de Engenharia de Software abordados ao longo do semestre. A transição de um modelo conceptual para uma arquitetura de software funcional demonstrou a importância de um design de classes rigoroso e de uma distribuição equilibrada de responsabilidades.

7.1 Conclusões

O balanço do projeto nesta fase é altamente positivo, destacando-se as seguintes conquistas estruturais:

- **Modularidade e Arquitetura:** A separação clara entre a camada de apresentação (Menus.cpp) e o controlador central (ManagerCatalogo.cpp) provou ser eficaz para manter o código organizado, legível e de fácil manutenção.
- **Aplicação de Princípios GRASP:** A atribuição de responsabilidades baseada nos padrões *Information Expert*, *Creator* e *Controller* evitou o acoplamento excessivo e garantiu que cada componente execute apenas as operações para as quais detém a informação necessária.
- **Robustez e Tolerância a Falhas:** A implementação de mecanismos de validação sintática nas entradas de dados e a criação de exceções personalizadas (InvalidDataException e InvalidRatingException) dotaram o sistema de uma barreira de proteção robusta, garantindo que o programa recupere de dados inválidos sem interrupções abruptas em tempo de execução.

Em suma, a arquitetura atual cumpre rigorosamente os requisitos de qualidade exigidos, resultando num software estável, limpo e preparado para expansões.

7.2 Trabalho Futuro (Iteração 3)

Como perspetiva de evolução e continuidade do projeto, o plano de desenvolvimento para a **Iteração 3** focar-se-á estritamente nos seguintes eixos operacionais:

- **Implementação Completa de Funcionalidades:** Conclusão e refinamento das operações secundárias do sistema, otimizando os algoritmos de pesquisa avançada por múltiplos critérios e consolidando os fluxos de persistência e remoção em lote.

- **Fase Intensiva de Testes (*Testing*):** Desenvolvimento e execução de um plano de testes sistemático para validar o comportamento do software. Esta fase incluirá testes de unidade para as funções lógicas do domínio (como o cálculo de totais no carrinho) e testes de integração de sistema para garantir que a interação entre a interface de utilizador e o controlador responde corretamente a cenários extremados ou inputs de stress no terminal.